Paladin Farm & Ranch

Developer Documentation

April 15, 2026

Table of Contents

1 Prerequisites

- Git or GitHub Desktop
- Node.js (includes npm)
- Docker

2 Setup

1. **Clone the repository**

```
git clone git@github.gatech.edu:cs-6150-computing-for-good/paladin.git
cd paladin
```

2. **Get the `.env` file** from the project mentor. It contains secrets for auth, database, Google Maps, PayPal, and push notifications. See Environment Variables for the full list. Do not commit this file.

3. **Install dependencies**

```
npm install
```

4. **Initialize the database** (starts PostgreSQL in Docker, runs migrations, seeds test data):

```
npm run init
```

5. **Start the development server**

```
npm run dev
```

Open http://localhost:3000 in your browser.

3 Test Accounts

Email	Password	Role
c4gdevad@gmail.com	*(ask mentor)*	ADMIN
c4gdevstaff@gmail.com	*(ask mentor)*	STAFF

You can also sign in with any personal Gmail account (it will have no role by default).

4 Installing as a PWA

Paladin is a Progressive Web App — users can install it on any device for a native app-like experience with push notifications and offline access to cached pages.

4.1 iOS (Safari)

1. Open the site in **Safari**
2. Tap the **Share** button (square with arrow)
3. Scroll down and tap **Add to Home Screen**
4. Tap **Add**

4.2 Android (Chrome)

1. Open the site in **Chrome**
2. Tap the **three-dot menu** in the top right
3. Tap **Add to Home Screen** (or **Install app** if prompted)
4. Tap **Install**

4.3 Desktop (Chrome / Edge)

1. Open the site in Chrome or Edge
2. Click the **install icon** (Monitor with arrow) in the address bar, or open the browser menu and select **Install Paladin Farm & Ranch**
3. Click **Install**

5 Useful Commands

Command	Description
`npm run dev`	Start dev server (Turbopack)
`npm run build`	Production build
`npm run lint`	Run ESLint
`npm run format:fix`	Auto-format with Prettier
`npm test`	Run Vitest tests
`npx prisma studio`	Open database GUI at localhost:5555
`npx prisma migrate dev`	Apply pending migrations
`make docs`	Generate user manual (PDF, Word, Markdown)

6 Configuration

All environment variables are defined in a `.env` file at the project root. A template is provided in `.env.example`. **Never commit actual values to version control** — `.env` is already in `.gitignore`.

7 Required Variables

7.1 Database (PostgreSQL)

Variable	Description
`DATABASE_URL`	Full connection string, e.g. `postgresql://user:password@localhost:5432/paladin`
`DATABASE_HOST`	Database hostname
`DATABASE_PORT`	Database port
`DATABASE_USER`	Database username
`DATABASE_PW`	Database password
`DATABASE_NAME`	Database name

7.2 NextAuth

Variable	Description
`AUTH_SECRET`	Session encryption key — generate with `openssl rand -base64 32`
`AUTH_URL`	Application URL, e.g. `http://localhost:3000`
`AUTH_GOOGLE_ID`	Google OAuth client ID
`AUTH_GOOGLE_SECRET`	Google OAuth client secret

7.3 Google Maps

Variable	Description
`NEXT_PUBLIC_GOOGLE_MAPS_API_KEY`	Google Maps JavaScript API key

7.4 Email (Resend)

Variable	Description
`RESEND_API_KEY`	API key for the Resend email service
`NOTIFICATION_EMAIL`	Recipient for contact form submissions

7.5 Push Notifications (VAPID)

Variable	Description
`NEXT_PUBLIC_VAPID_PUBLIC_KEY`	Public VAPID key for web push
`VAPID_PRIVATE_KEY`	Private VAPID key

7.6 PayPal

Variable	Description
`NEXT_PUBLIC_PAYPAL_CLIENT_ID`	PayPal client ID (exposed to browser)
`NEXT_PUBLIC_PAYPAL_PLAN_ID`	PayPal subscription plan ID
`PAYPAL_CLIENT_ID`	PayPal client ID (server-side)
`PAYPAL_SECRET`	PayPal API secret
`PAYPAL_API_BASE`	API base URL — defaults to `https://api-m.sandbox.paypal.com`

8 Obtaining Credentials

Contact the project mentor for the complete `.env` file. For new API keys:

- **Google OAuth**: Google Cloud Console → APIs & Services → Credentials
- **Google Maps**: Same console → enable Maps JavaScript API
- **Resend**: resend.com/api-keys
- **PayPal**: developer.paypal.com → My Apps & Credentials
- **VAPID keys**: Generate with `npx web-push generate-vapid-keys`

9 Overview

Paladin Farm & Ranch is a full-stack Next.js application that connects farmers affected by natural disasters with nearby volunteers and organizations. Users register farms, create emergency requests, and coordinate responses through a real-time map-based dashboard.

10 Tech Stack

Layer	Technology
Framework	Next.js 16 (App Router, React 19, Turbopack)

Layer	Technology
Language	TypeScript
Database	PostgreSQL
ORM	Prisma
Auth	NextAuth.js v5 (Google OAuth)
Styling	Tailwind CSS
Maps	Google Maps JavaScript API
Payments	PayPal Subscriptions API
Email	Resend
Push Notifications	Web Push (VAPID)
Docs	Fumadocs (MDX)
UI Components	Radix UI, shadcn/ui
Diagrams	Mermaid

11 Dependencies

11.1 Core

Package	Version	Purpose
`next`	^16.1.6	React framework (App Router)
`react` / `react-dom`	19.2.1	UI library
`typescript`	^5.7.3	Type safety
`@prisma/client`	^6.4.1	Database ORM
`next-auth`	^5.0.0-beta.25	Authentication (Google OAuth)
`@auth/prisma-adapter`	^2.7.4	NextAuth + Prisma integration

11.2 UI

Package	Version	Purpose
`tailwindcss`	^3.4.1	Utility-first CSS
`lucide-react`	^0.464.0	Icons
`ag-grid-react`	^33.0.3	Admin data tables
`@react-google-maps/api`	^2.20.6	Google Maps
`next-themes`	^0.4.4	Dark / light mode
`@radix-ui/*`	various	Headless UI primitives (11 packages)
`mermaid`	^11.14.0	Client-side diagram rendering

11.3 Forms and Validation

Package	Version	Purpose
`react-hook-form`	^7.54.2	Form state management
`@hookform/resolvers`	^4.1.3	Zod integration for react-hook-form
`zod`	^3.24.2	Schema validation

11.4 Services

Package	Version	Purpose
`resend`	^6.9.2	Transactional email
`web-push`	^3.6.7	Browser push notifications
`@paypal/react-paypal-js`	^8.9.2	Donations and subscriptions

11.5 Documentation

Package	Version	Purpose
`fumadocs-core`	^16.7.10	In-app documentation framework
`fumadocs-mdx`	^14.2.11	MDX processing

11.6 Development Tools

Package	Version	Purpose
`vitest`	^3.1.1	Unit testing
`eslint` / `eslint-config-next`	^9 / ^16.1.6	Linting
`prettier`	^3.4.2	Code formatting
`husky`	^9.1.7	Git hooks
`lint-staged`	^15.2.11	Run linters on staged files only
`prisma`	^6.4.1	Database migrations and tooling
`@mermaid-js/mermaid-cli`	^11.12.0	Diagram PNG rendering for PDF docs
Docker	—	Local PostgreSQL container

12 Architecture Diagram

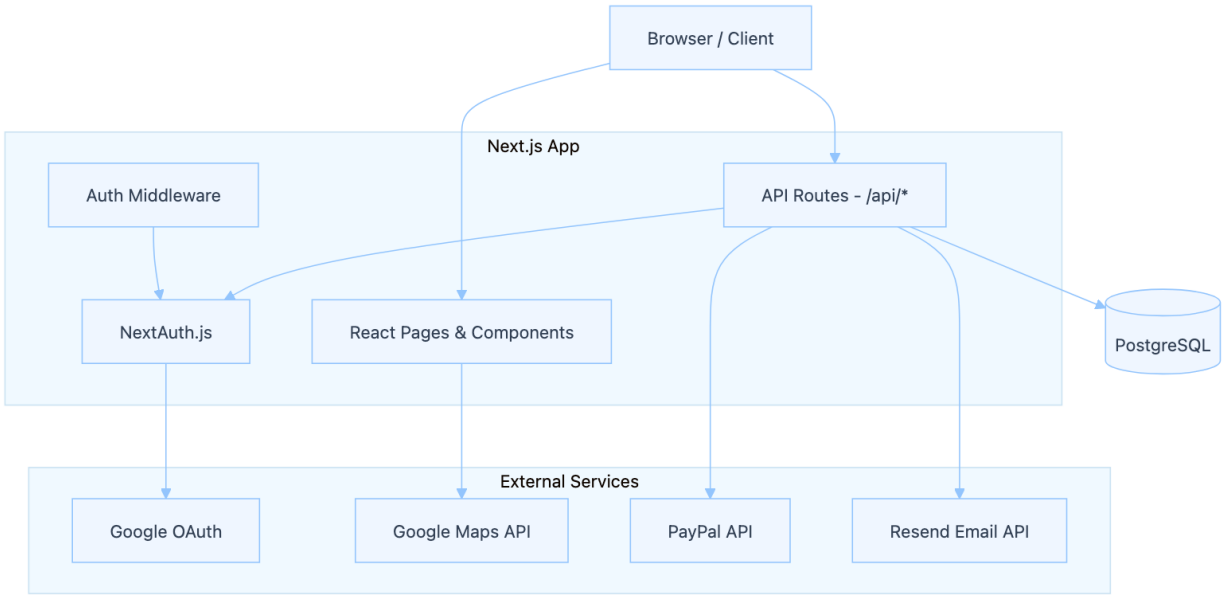

Figure 1: Architecture Diagram

13 Key Directories

```
src/
  app/                    # Next.js App Router
    api/                  # API route handlers
      requests/           # Emergency request CRUD
      organizations/      # Org membership & management
      users/              # User management (admin)
      farms/              # Farm CRUD
      paypal/             # PayPal orders & subscriptions
      ...
    dashboard/            # Map-based dashboard
    organizations/        # Org browsing & management
    profile/              # User profile editing
    registration/         # Multi-step registration
  components/             # React components
  lib/                    # Shared utilities
    auth.ts               # NextAuth config
    prisma.ts             # Prisma client singleton
    email.ts              # Email templates (Resend)
    paypal-subscriptions.ts
  hooks/                  # Custom React hooks
prisma/
  schema.prisma           # Database schema
  migrations/             # Migration history
  seed.mjs                # Seed data
content/docs/             # MDX documentation pages
```

14 Current Production Setup

Paladin is currently deployed at paladinfarmandranch.com on infrastructure provided by Georgia Tech. The application runs as a standalone Next.js build behind Nginx, with a PostgreSQL database on the same server. GitHub Actions handles CI/CD (see `.github/workflows/cd.yaml`).

15 Recommended Free Hosting

If the GT-provided server is no longer available, the following free-tier options are suitable:

Component	Recommended	Notes
Database	Neon or Supabase	Free PostgreSQL hosting; no credit card required
Application	Vercel	Free tier for Next.js apps; automatic deployments from GitHub. Next.js standalone output is already configured (`output: 'standalone'` in `next.config.mjs`)

Component	Recommended	Notes
Alternative	Render	Free tier for web services + PostgreSQL if a single provider is preferred

15.1 Migration Notes

- Update `DATABASE_URL` in environment variables to point to the new hosted PostgreSQL instance
- Run `npx prisma migrate deploy` against the new database to apply the schema
- Set all environment variables (see Environment Variables) on the hosting platform
- Vercel auto-detects Next.js projects — connect the GitHub repo and it deploys on push

16 Authentication

Authentication uses **NextAuth.js v5** with the **Google OAuth** provider. Configuration is in `src/lib/auth.ts`.

Key files:

File	Purpose
`auth.ts`	NextAuth config — Google provider, Prisma adapter, session callbacks
`[...nextauth]/route.ts`	Auth API route handler
`signin/page.tsx`	Custom sign-in page
`schema.prisma`	User, Account, and Session models (Prisma adapter)

16.1 How It Works

1. User clicks **Sign In** → redirected to Google OAuth consent screen
2. On success, NextAuth creates/updates `User` and `Account` records via the Prisma adapter
3. A session is created and stored in the `Session` table
4. The `auth()` helper (exported from `src/lib/auth.ts`) is used in API routes and server components to check the session

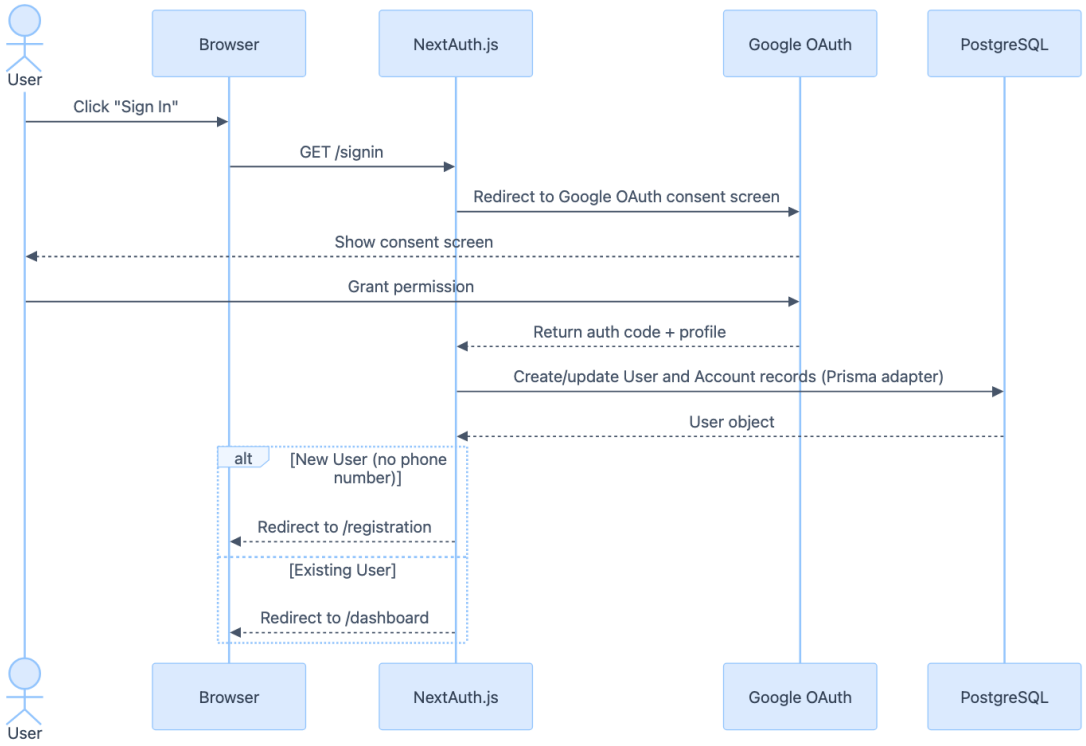

Figure 2: How It Works

16.2 Registration Flow

New users are redirected to a multi-step registration form. Each step collects different information and POSTs to the respective API route.

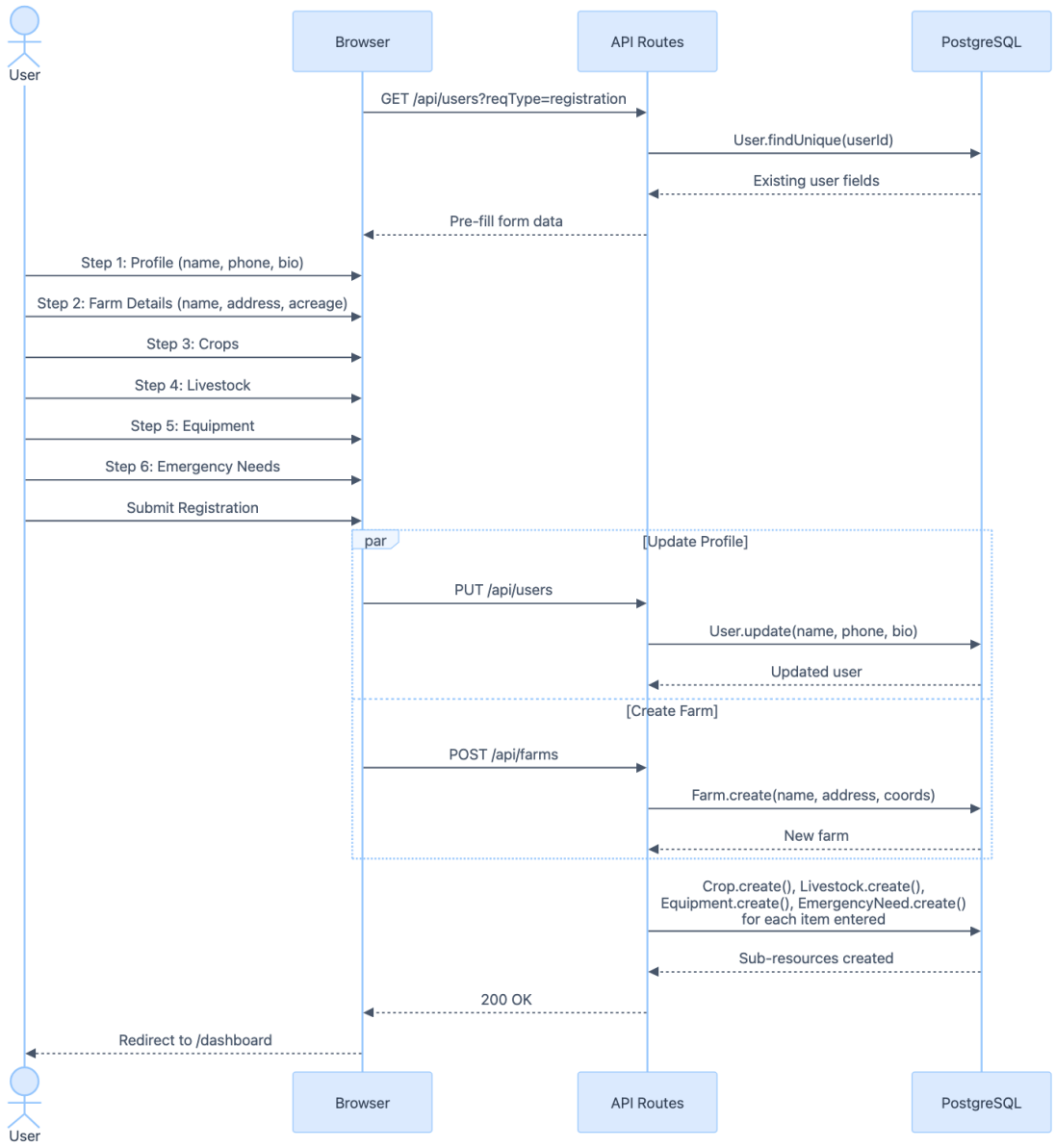

Figure 3: Registration Flow

16.3 Account Linking

The config uses `allowDangerousEmailAccountLinking: true` so that Google accounts merge with email-matched records. Since only Google sign-up is supported, this enables seeded test accounts to work with OAuth.

17 Authorization

There are two levels of roles:

17.1 Platform Roles (`UserRole`)

Stored on the `User` model. Defined in `prisma/schema.prisma`:

Role	Access
`ADMIN`	Full access — manage users, review org requests, manage resources, bypass subscription checks
`STAFF`	Bypass subscription checks for request creation
(none)	Regular user — standard access

17.2 Organization Roles (`OrgRole`)

Stored on `OrganizationMember`. Controls per-org permissions:

Role	Access
`OWNER`	Full org control — manage members, approve/reject join requests, delete org
`MANAGER`	Manage members and join requests (cannot remove owners)
`MEMBER`	View-only org membership

17.3 Where Authorization Is Checked

- **API routes** — each route handler calls `auth()` and checks `session.user.role` or org membership via `checkOrgAdminAccess()` in `src/app/api/organizations/[id]/members/route.ts`
- **Client components** — conditional UI rendering based on `session.user.role` and the user's org role from API responses

18 Backup

The database is PostgreSQL, run via Docker in development. Use `pg_dump` inside the container to create a backup:

```
docker exec paladin-paladin-db-1 pg_dump -U root paladin > backup.sql
```

19 Restore

To restore a backup into the local Docker database:

```
docker exec -i paladin-paladin-db-1 psql -U root paladin < backup.sql
```

For a remote/production database:

```
psql -U $DATABASE_USER -h $DATABASE_HOST -p $DATABASE_PORT $DATABASE_NAME < backup.sql
```

Or rebuild from schema (no data):

```
npx prisma migrate deploy
npx prisma db seed
```

20 Schema Files

File	Purpose
`prisma/schema.prisma`	Prisma schema definition (models, relations, enums)
`prisma/migrations/`	Ordered SQL migration files
`prisma/seed.mjs`	Seed script with test data
`prisma/seed/`	Seed data modules (users, farms, organizations, etc.)

21 Automated Backups

In production, database backups should be scheduled via cron or the hosting provider's backup feature. If using Neon or Supabase, automatic point-in-time recovery is included on their free tiers.

22 Overview

The Prisma schema defines the application's data model backed by PostgreSQL. The schema is located at `prisma/schema.prisma` and covers user accounts, farms, emergency requests/responses, organizations, and supporting entities.

23 Class Diagram

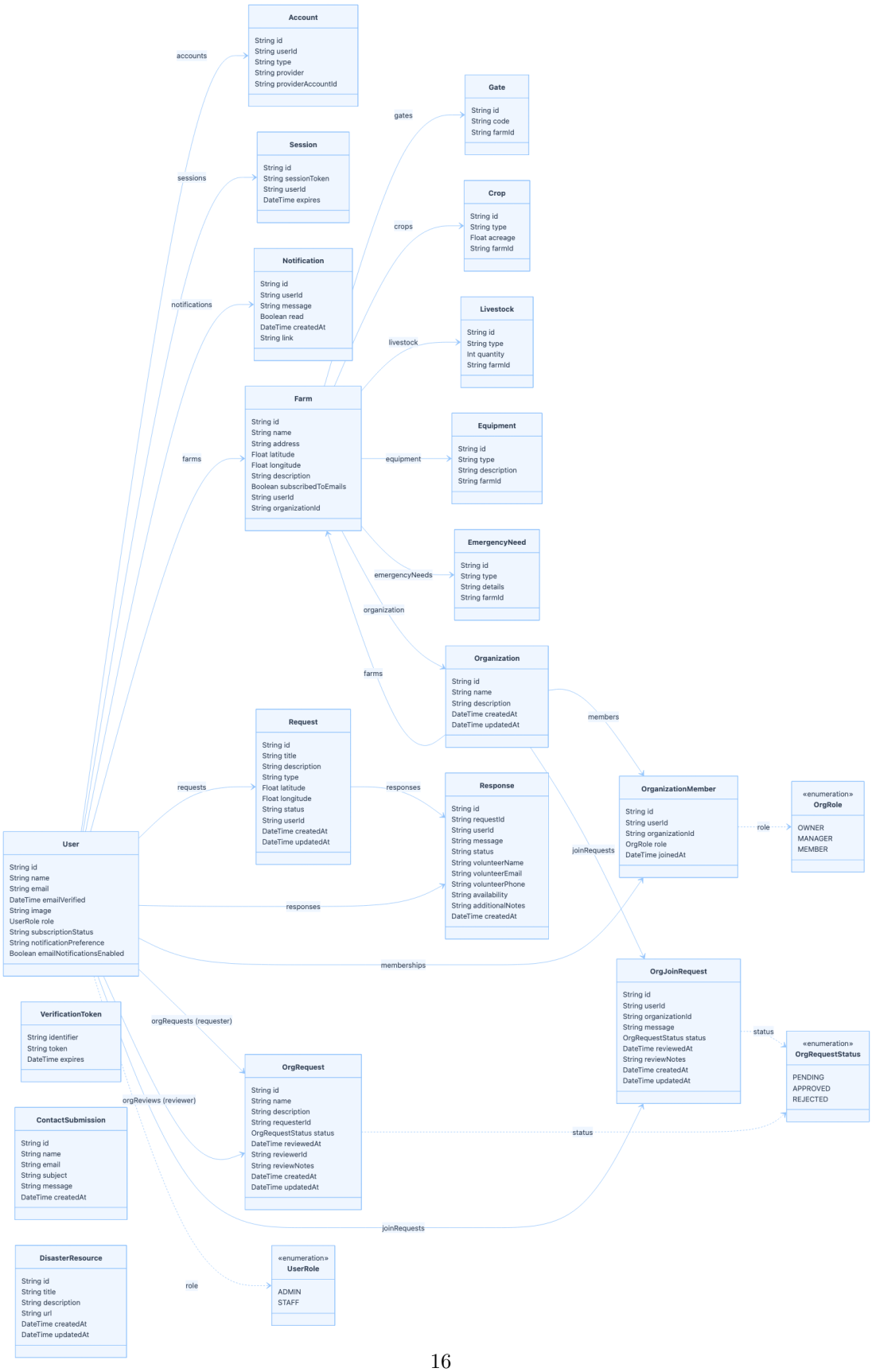

Figure 4: Class Diagram

24 Model Groups

24.1 Authentication & Identity

- **User** — Central model. Stores profile info, role (`ADMIN` / `STAFF`), subscription status, and notification preferences. All other user-scoped models reference this.
- **Account** — OAuth provider link (Google). One user can have multiple provider accounts.
- **Session** — Active browser session with expiry. Managed by NextAuth.
- **VerificationToken** — Email verification tokens (NextAuth).

24.2 Farm & Assets

- **Farm** — Registered farm with address, GPS coordinates, and optional organization membership. Owned by one `User`.
- **Gate** — Named gate/access point on a farm.
- **Crop** — Crop type and acreage.
- **Livestock** — Livestock type and count.
- **Equipment** — Farm equipment entries.
- **EmergencyNeed** — Specific needs declared during an emergency request.

24.3 Requests & Responses

- **Request** — Emergency assistance request with type, status, and map coordinates. Created by a `User`.
- **Response** — Volunteer response to a request, including contact details, availability, and status.

24.4 Organizations

- **Organization** — Group that farms can belong to. Has members and incoming join requests.
- **OrganizationMember** — Join table linking `User` to `Organization` with an `OrgRole` (`OWNER` / `MANAGER` / `MEMBER`).
- **OrgRequest** — Request to create a new organization. Reviewed by an admin.
- **OrgJoinRequest** — Request from a user to join an existing organization.

24.5 Standalone

- **ContactSubmission** — Public contact-us form submissions.
- **DisasterResource** — Admin-curated disaster preparedness resources.
- **Notification** — In-app notifications with read status and optional deep-link.

25 Enumerations

Enum	Values	Used By
`UserRole`	`ADMIN`, `STAFF`	`User.role`
`OrgRole`	`OWNER`, `MANAGER`, `MEMBER`	`OrganizationMember.role`
`OrgRequestStatus`	`PENDING`, `APPROVED`, `REJECTED`	`OrgRequest.status`, `OrgJoinRequest.status`

26 Diagrams

27 Emergency Requests

27.1 Creating an Emergency Request

1. User fills out the form in `src/components/DashboardPage.tsx`
2. Client POSTs to `src/app/api/requests/route.ts`
3. API validates auth session, checks PayPal subscription status via `src/lib/paypal-subscriptions.ts`
4. Prisma creates the `Request` record in PostgreSQL
5. Push notifications sent to nearby farm owners via `src/app/api/requests/route.ts` → `notify()`
6. Confirmation email sent via `src/lib/email.ts` → Resend API

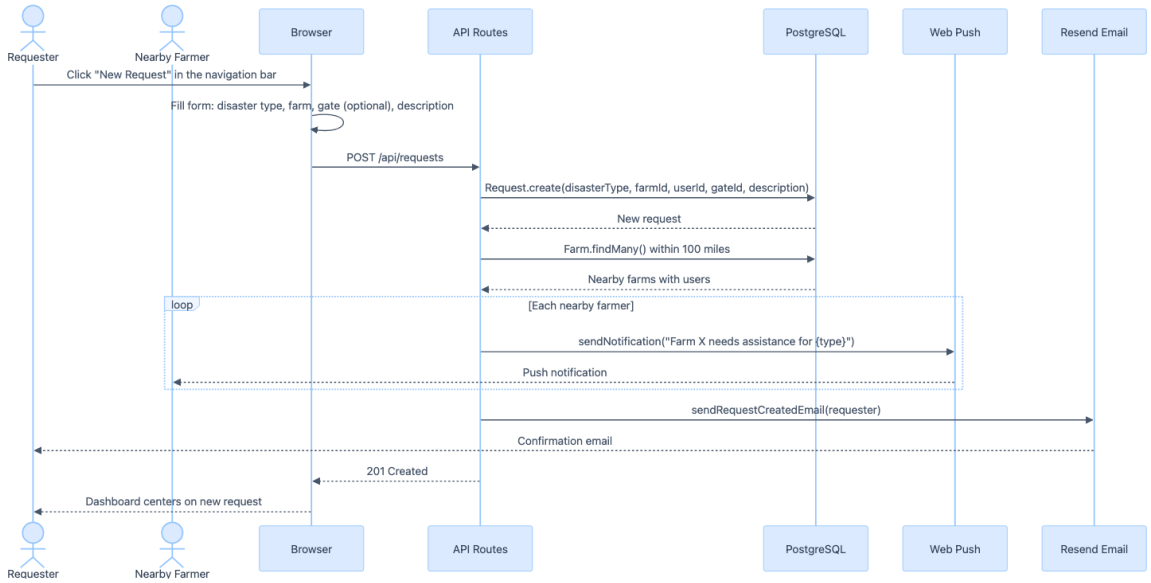

Figure 5: Creating an Emergency Request

27.2 Viewing Requests

Requests are fetched based on map bounds and filtered by visibility rules depending on user role and sidebar tab.

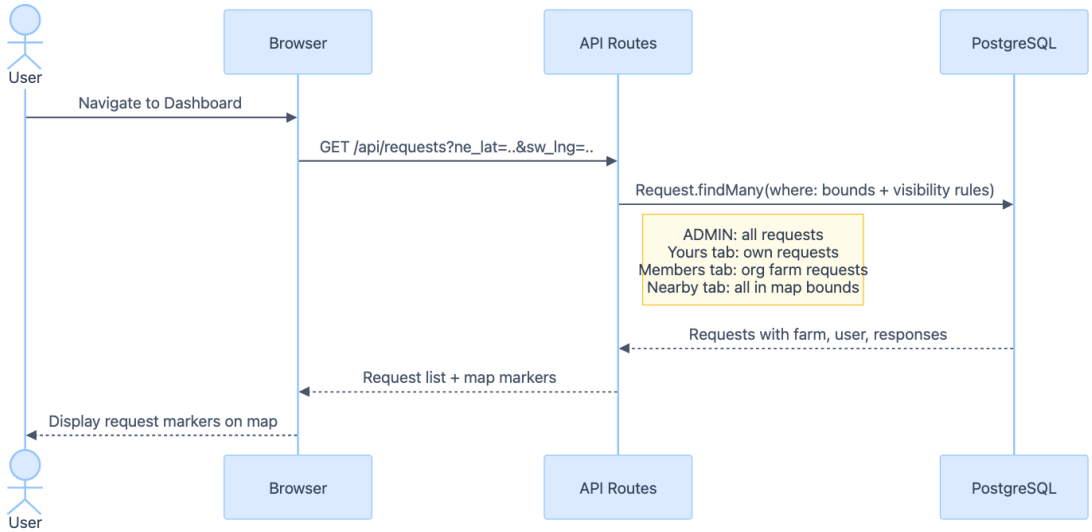

Figure 6: Viewing Requests

27.3 Responding to a Request

1. User views request details in `src/components/DashboardPage.tsx`
2. Client POSTs response to `src/app/api/requests/[requestId]/respond/route.ts`
3. Request owner notified by email via `src/lib/email.ts`

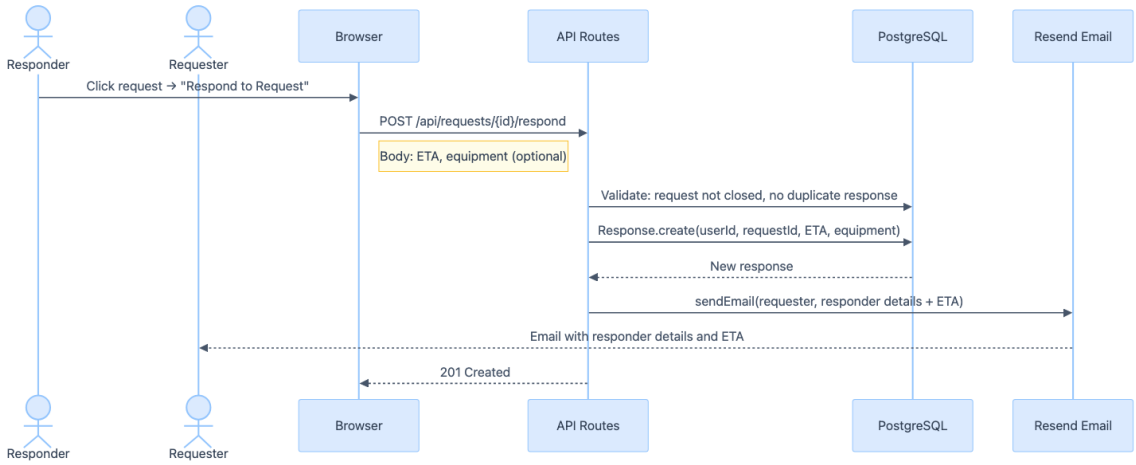

Figure 7: Responding to a Request

27.4 Cancelling a Response

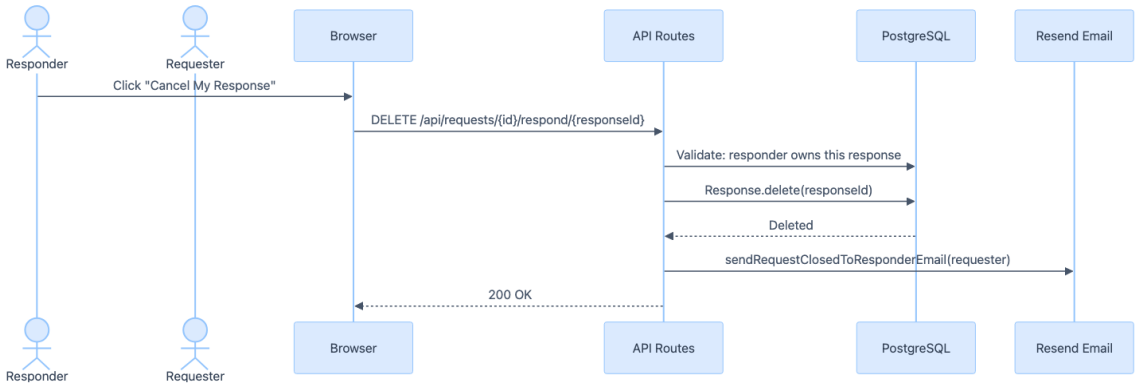

Figure 8: Cancelling a Response

27.5 Closing a Request

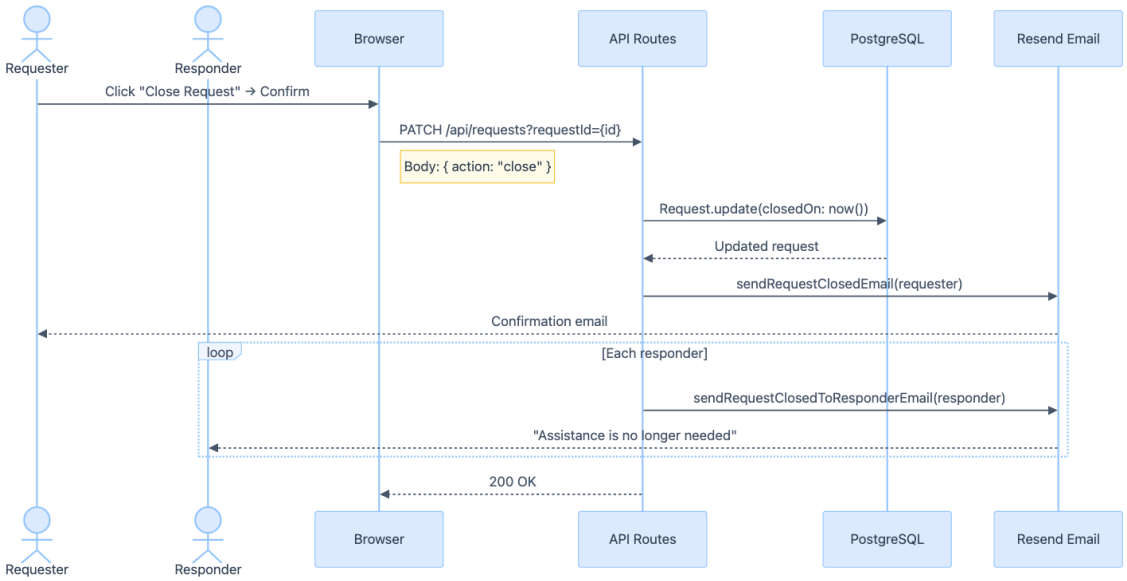

Figure 9: Closing a Request

28 Organizations

28.1 Organization Creation

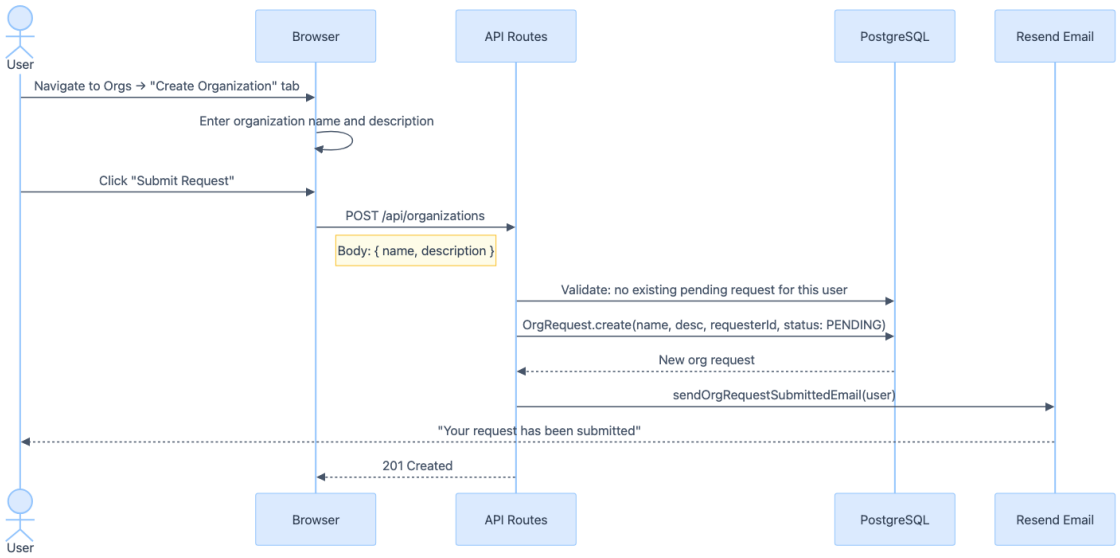

Figure 10: Organization Creation

28.2 Admin: Approve or Reject Organization

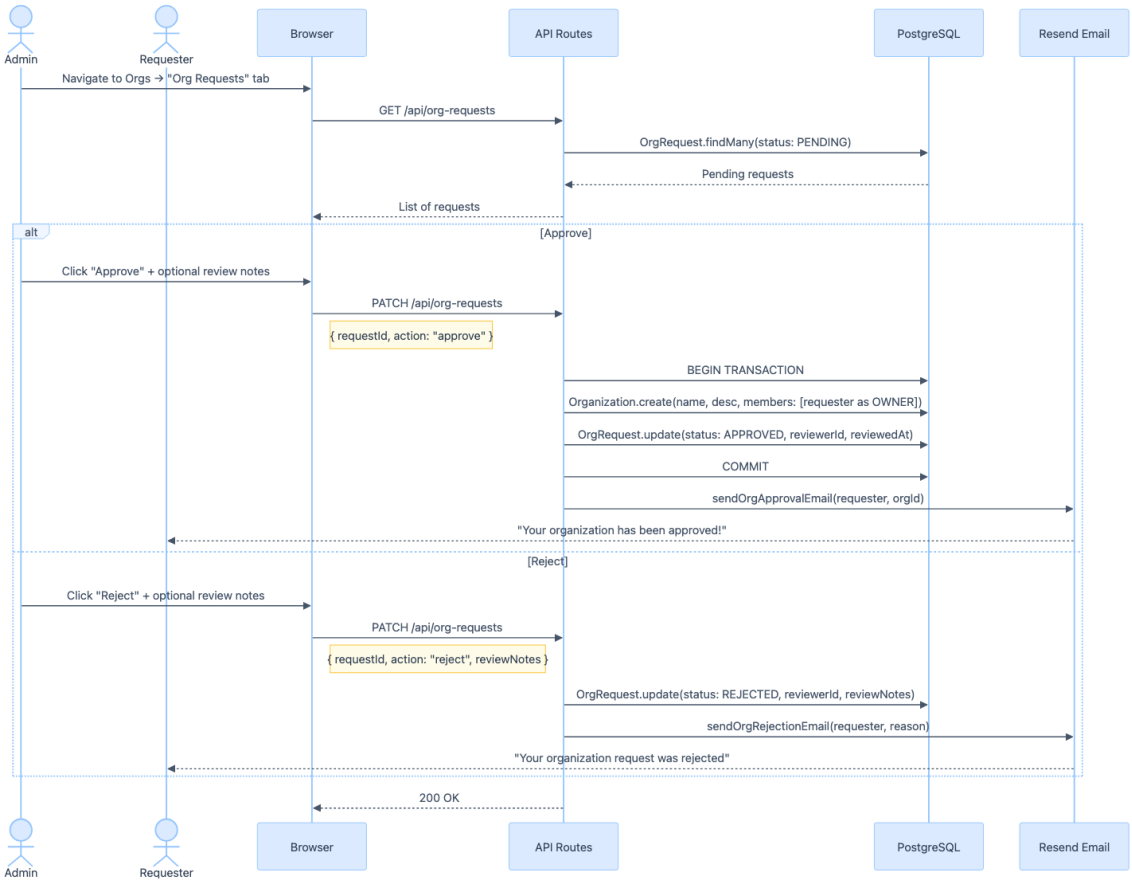

Figure 11: Admin: Approve or Reject Organization

28.3 Joining an Organization

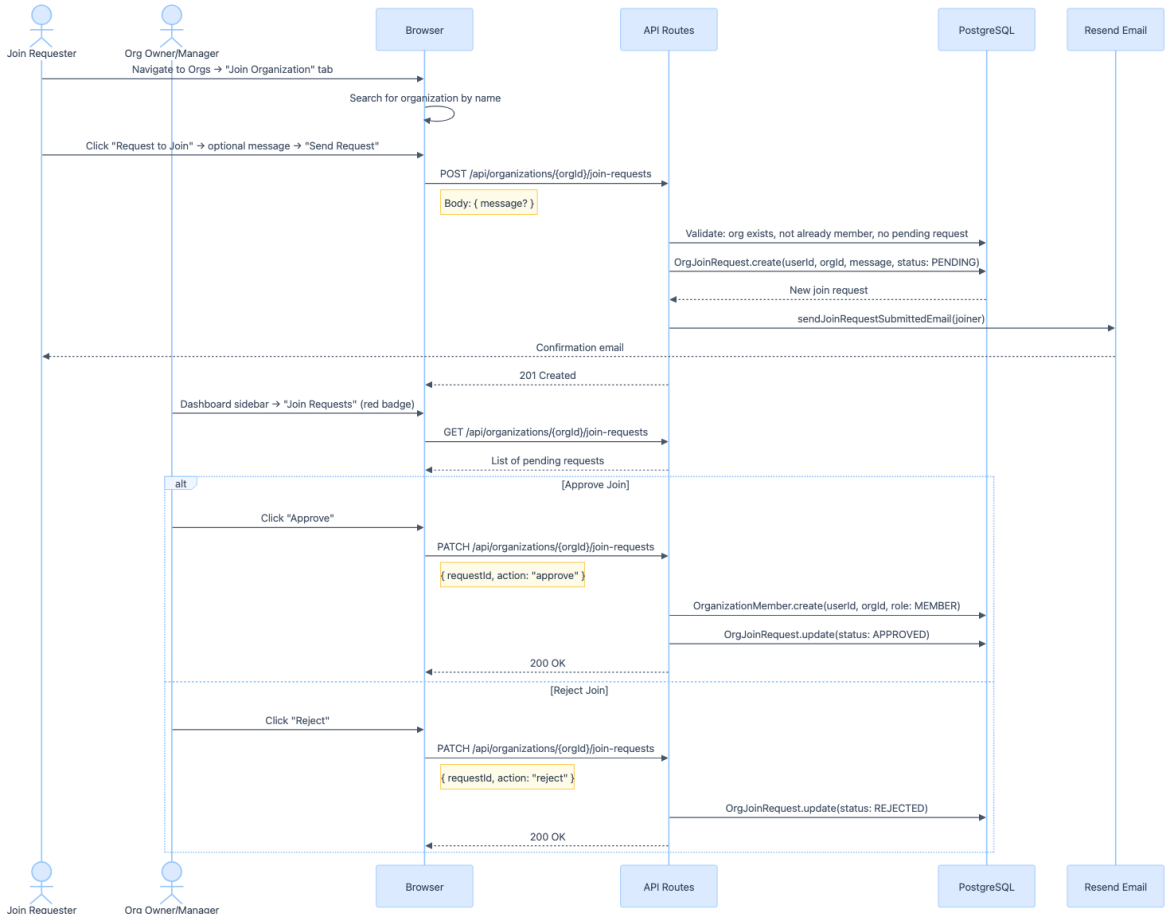

Figure 12: Joining an Organization

28.4 Managing Organization Members

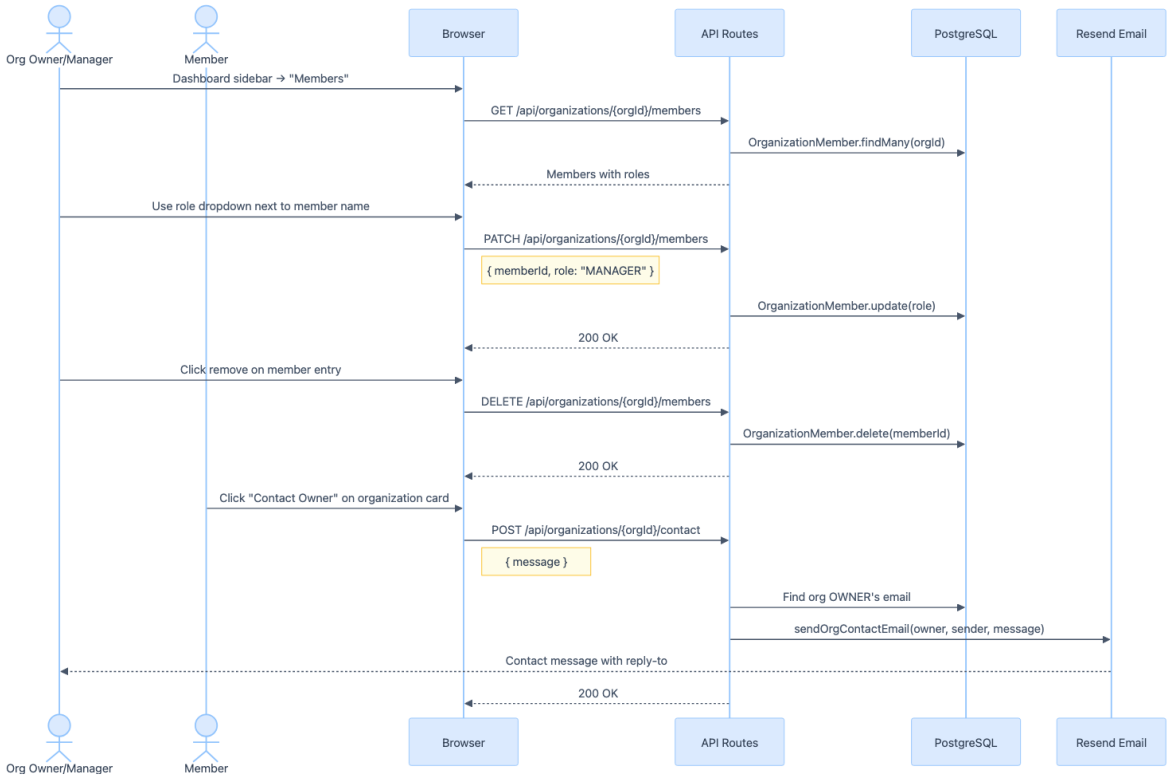

Figure 13: Managing Organization Members

29 Profile and Farm Management

29.1 View and Update Profile

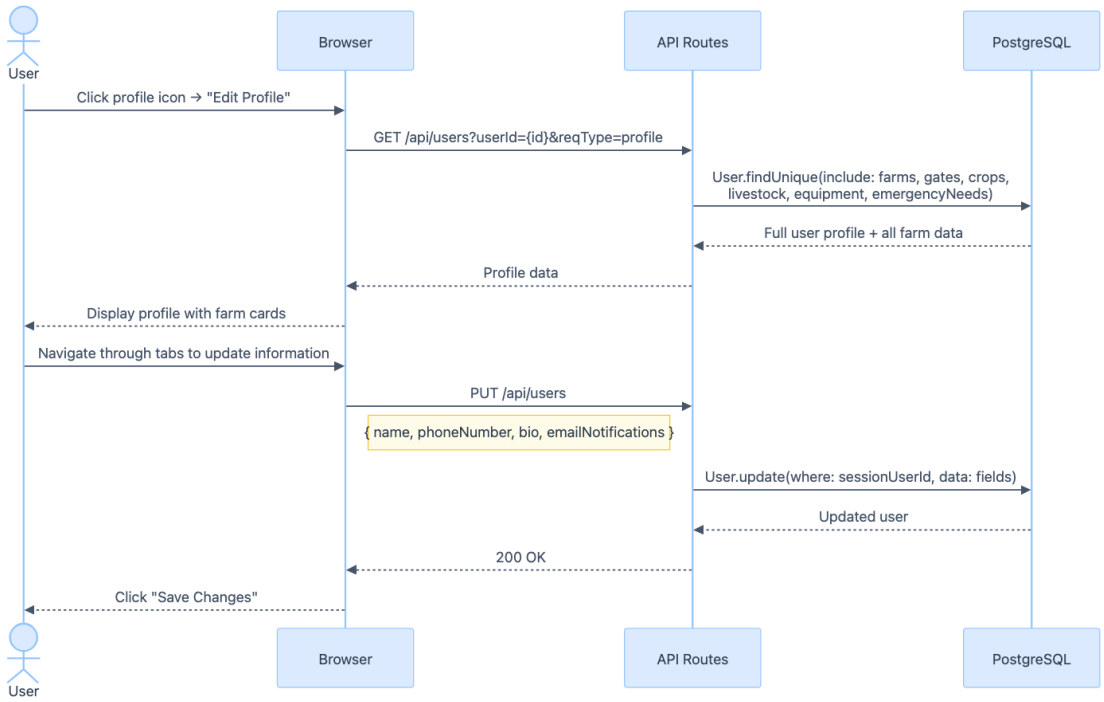

Figure 14: View and Update Profile

29.2 Farm CRUD

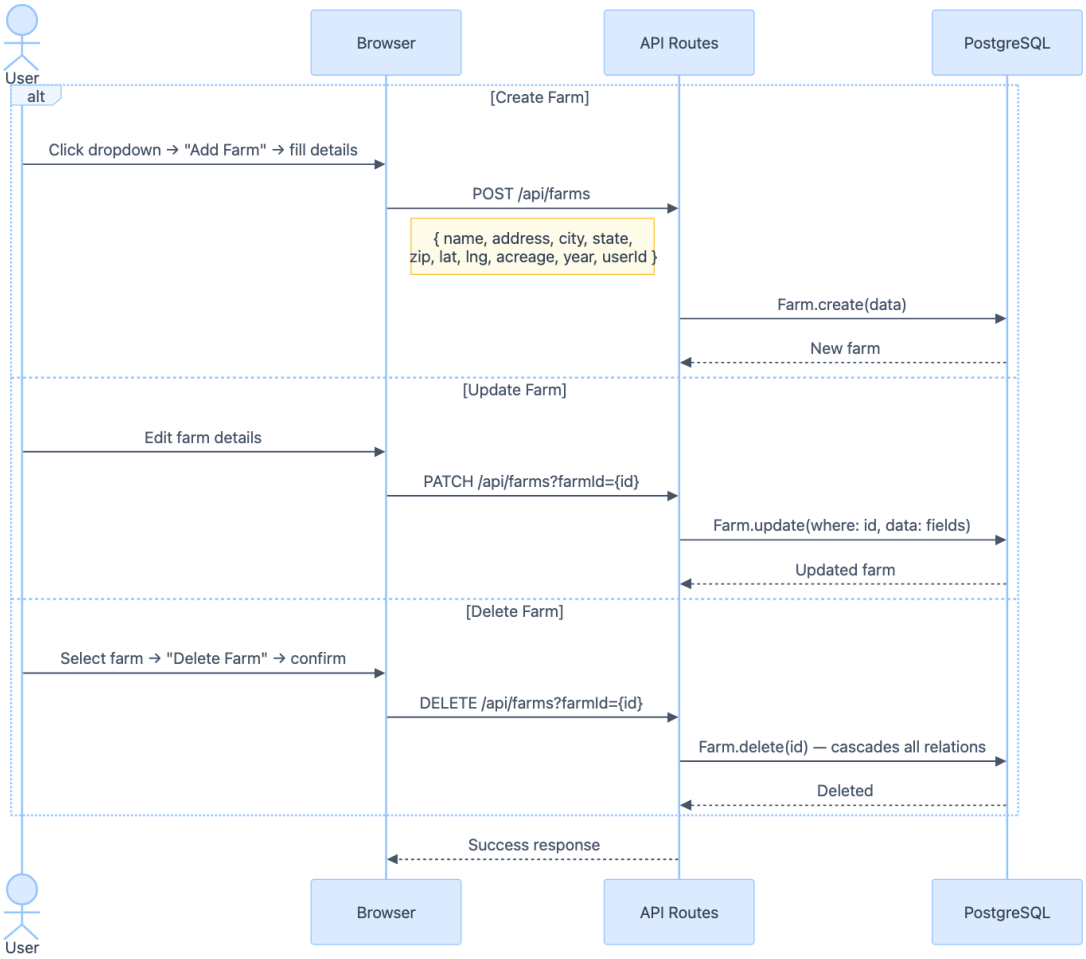

Figure 15: Farm CRUD

29.3 Farm Sub-Resources (Crops, Livestock, Equipment, Gates, Emergency Needs)

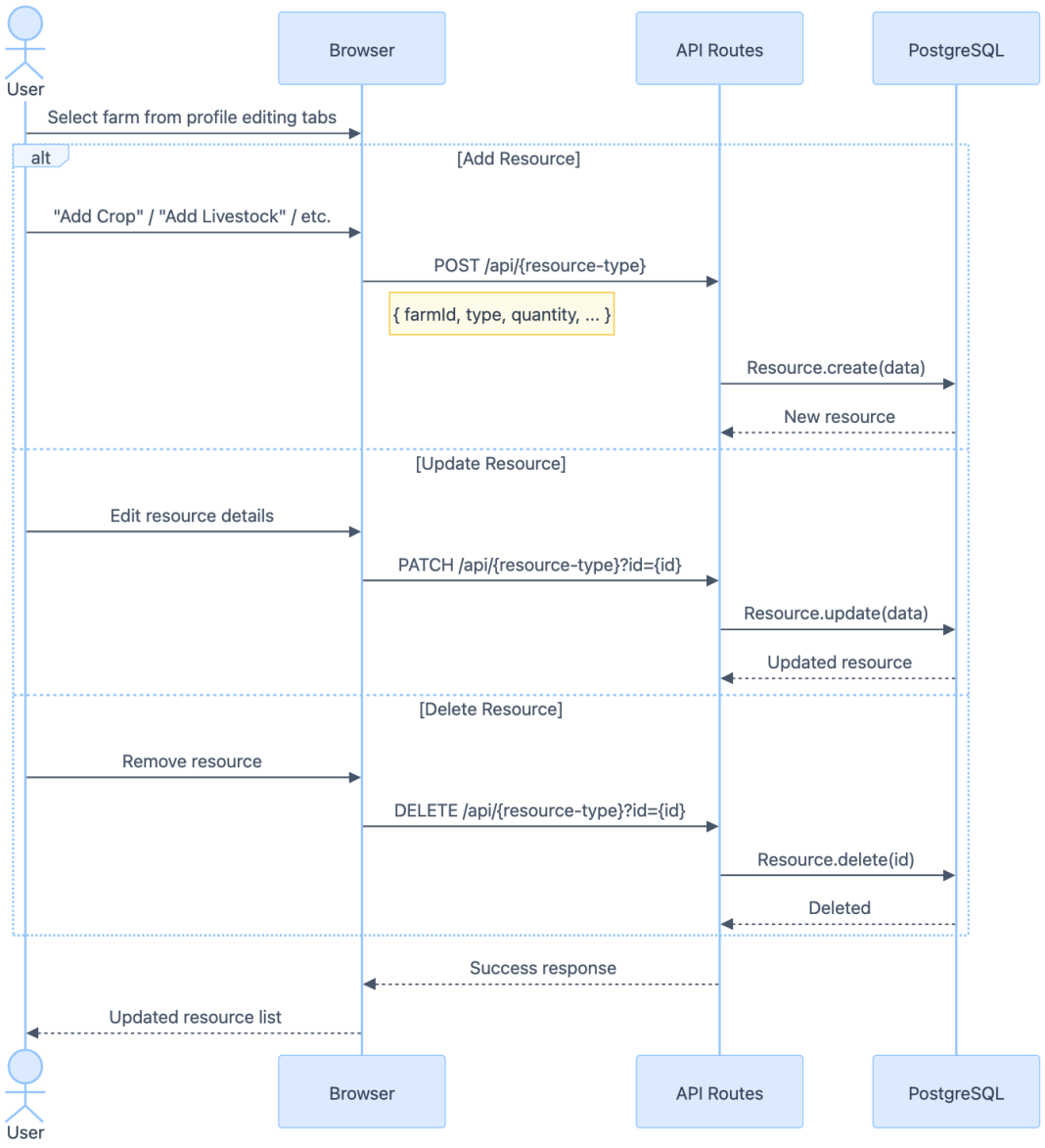

Figure 16: Farm Sub-Resources (Crops, Livestock, Equipment, Gates, Emergency Needs)

30 Admin and System

30.1 Admin: User Management

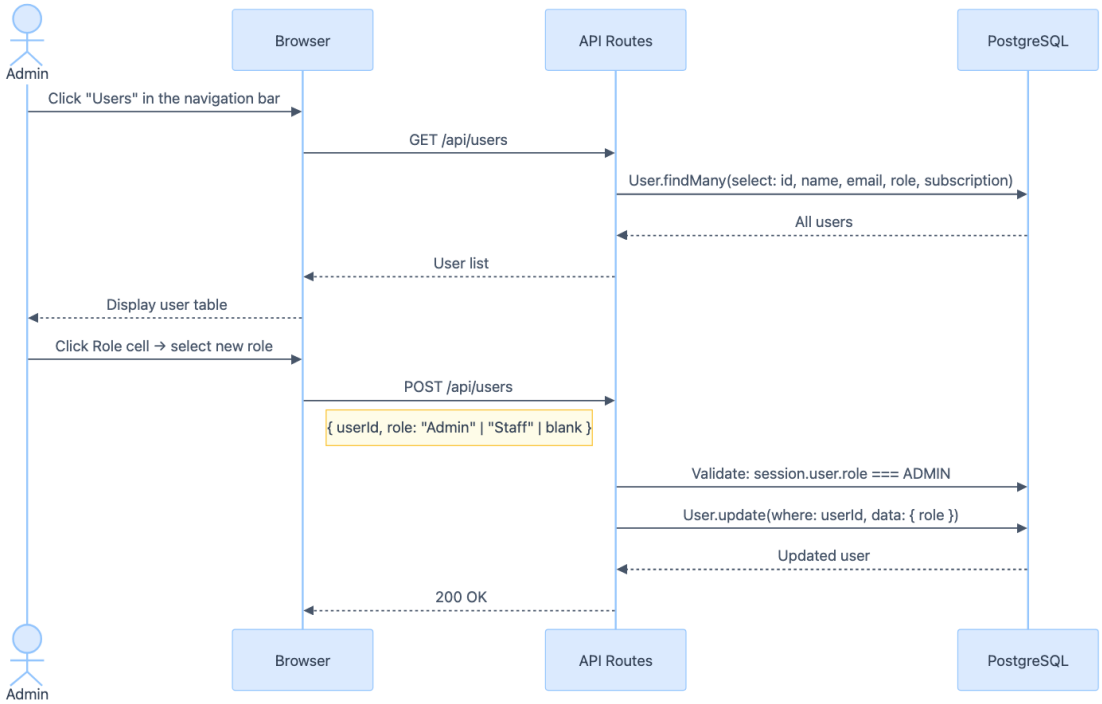

Figure 17: Admin: User Management

30.2 Admin: Disaster Resources

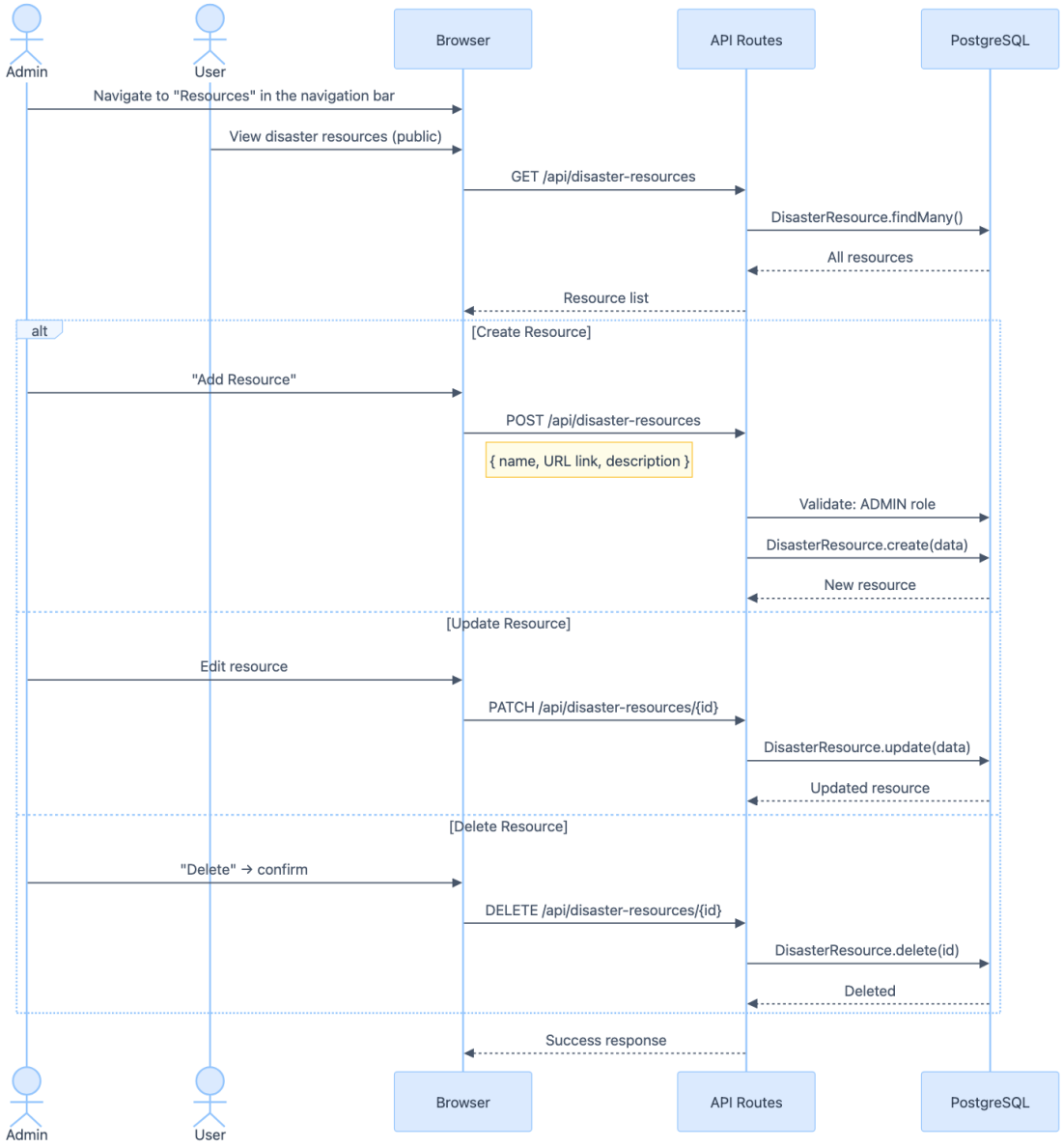

Figure 18: Admin: Disaster Resources

30.3 Push Notification Registration

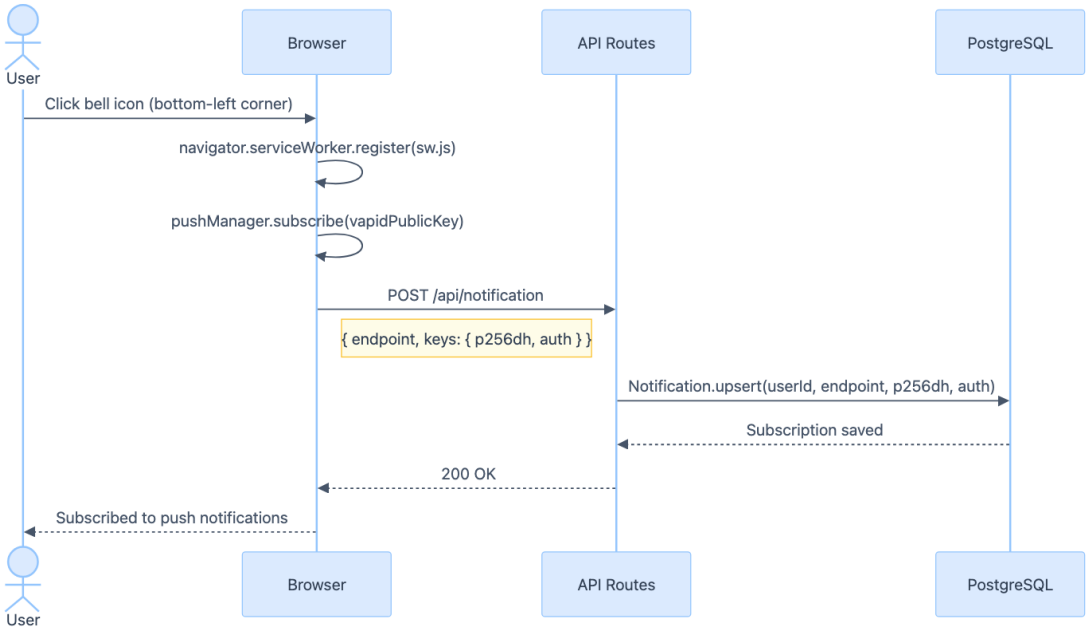

Figure 19: Push Notification Registration

30.4 Contact Us

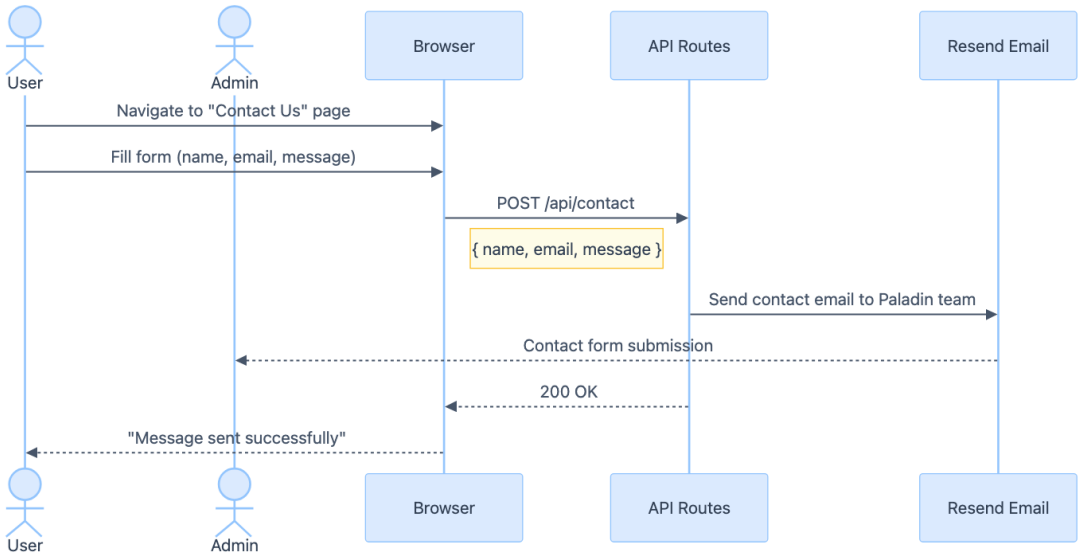

Figure 20: Contact Us